4

macro language and the macro processor

The assembly language programmer often finds it necessary to repeat some blocks of code many times in the course of a program. The block may consist of code to save or exchange sets of registers, for example, or code to set up linkages or perform a series of arithmetic operations. In this situation the programmer will find a macro instruction facility useful. *Macro instructions* (often called *macros*) are single-line abbreviations for groups of instructions. In employing a macro, the programmer essentially defines a single "instruction" to represent a block of code. For every occurrence of this one-line macro instruction in his program, the macro processing assembler will substitute the entire block.

By defining the appropriate macro instructions, an assembly language programmer can tailor his own higher level facility in a convenient manner, at no cost in control over the structure of his program. He can achieve the conciseness and ease in coding of high level languages without losing the basic advantage of assembly language programming. Integral macro operations simplify debugging and program modification and they facilitate standardization. Many computer manufacturers use macro instructions to automate the writing of "tailored" operating systems in a process called *systems generation.*

Macro instructions are usually considered an extension of the basic assembler language, and the macro processor is viewed as an extension of the basic assembler algorithm. As a form of programming language, however, macro instruction languages differ significantly from assembly languages and compiled algebraic languages. Important analogs are to be found in some high level languages and text editing systems.

In this section we will discuss an assembler macro facility. We will treat the use of macro instructions in programming and the implementation of macros within an assembler. Certain features that are characteristic of most macro processors will be introduced and developed.

©Rimmu_notes_pdf

4.1 MACRO INSTRUCTIONS

In its simplest form, a macro is an abbreviation for a sequence of operations. Consider the following program:

Example 1:

```
        ⋮
A       1, DATA     Add contents of DATA to register 1
A       2, DATA     Add contents of DATA to register 2
A       3, DATA     Add contents of DATA to register 3
        ⋮
A       1, DATA     Add contents of DATA to register 1
A       2, DATA     Add contents of DATA to register 2
A       3, DATA     Add contents of DATA to register 3
        ⋮
DATA    DC      F'5'
        ⋮
```

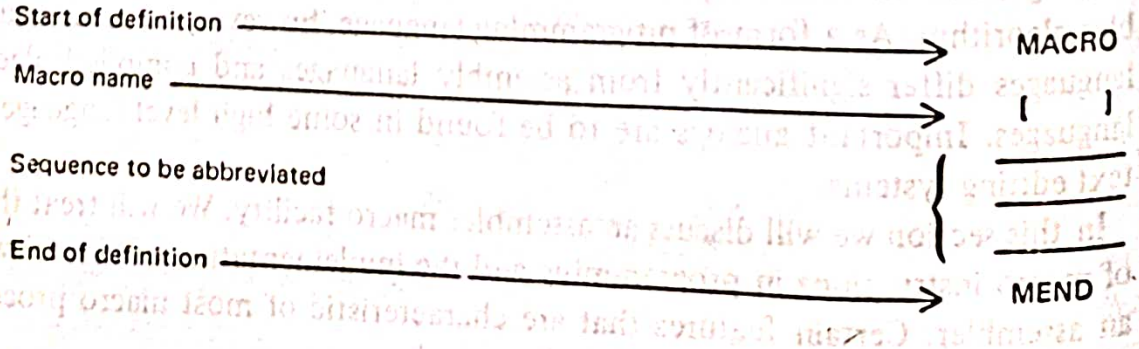
In the above program the sequence

```
A       1, DATA
A       2, DATA
A       3, DATA
```

occurs twice. A macro facility permits us to attach a name to this sequence and to use this name in its place. We can invent a macro language that allows us to specify the above as a macro definition and allows us to refer to the definition later. We have chosen to use a 360-type macro language.

A macro processor effectively constitutes a separate language processor with its own language.

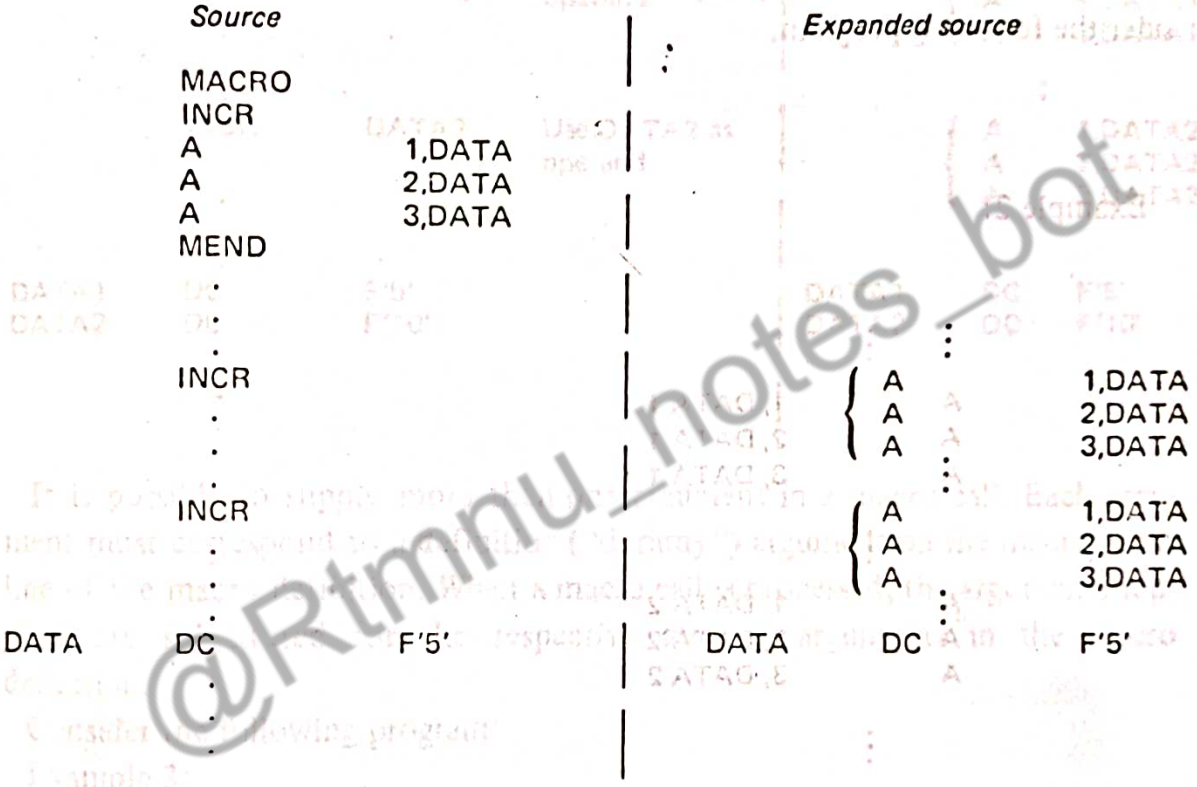
We attach a name to a sequence by means of a macro instruction definition, which is formed in the following manner:



@Ptmu_notes_bot

The MACRO pseudo-op is the first line of the definition and identifies the following line as the *macro instruction name.* Following the name line is the sequence of instructions being abbreviated–the instructions comprising the 'macro' instruction. The definition is terminated by a line with the MEND ('macro end') pseudo-op.

Once the macro has been defined, the use of the macro name as an operation mnemonic in an assembly program is equivalent to the use of the corresponding instruction sequence. Our example might be rewritten as follows, assigning the name 'INCR' to the repeated sequence.



In this case the macro processor replaces each macro call with the lines

```
A       1, DATA
A       2, DATA
A       3, DATA
```

This process of replacement is called *expanding* the macro. Notice that the macro definition itself does not appear in the expanded source code. The definition is saved by the macro processor. The occurrence in the source program of the macro name, as an operation mnemonic to be expanded, is called a *macro call.*

@Rtmnu_notes_bot

4.2 FEATURES OF A MACRO FACILITY

4.2.1 Macro Instruction Arguments

The macro facility presented thus far is capable of inserting blocks of instructions in place of macro calls. All of the calls to any given macro will be replaced by identical blocks. This macro facility lacks flexibility: there is no way for a specific macro call to modify the coding that replaces it. An important extension of this facility consists of providing for arguments, or parameters, in macro calls. Corresponding *macro dummy arguments* will appear in macro definitions. Consider the following program.

Example 2:

```
          .
          .
          .
          A       1, DATA 1
          A       2, DATA 1
          A       3, DATA 1
          .
          .
          .
          A       1, DATA 2
          A       2, DATA 2
          A       3, DATA 2
          .
          .
          .
DATA 1    DC      F'5'
DATA 2    DC      F'10'
```

In this case the instruction sequences are very similar but not identical. The first sequence performs an operation using DATA1 as operand; the second, using DATA2. They can be considered to perform the same operation with a variable parameter, or argument. Such a parameter is called a *macro instruction argument*, or "dummy argument;" it is specified on the macro name line and distinguished (as a macro language symbol rather than an assembly language symbol) by the ampersand (&), which is always its first character. The preceding program could be written as:

@Rtmnu_notes_bot

Source			Expanded source		
MACRO		Macro INCR has one argument			
INCR	&ARG				
A	1,&ARG				
A	2,&ARG				
A	3,&ARG				
MEND					
.					
.					
.					
INCR	DATA1	Use DATA1 as operand			
			A	1,DATA1	
			A	2,DATA1	
			A	3,DATA1	
.					
.					
.					
INCR	DATA2	Use DATA2 as operand			
			A	1,DATA2	
			A	2,DATA2	
			A	3,DATA2	
.					
.					
.					
DATA1 DC	F'5'		DATA1	DC	F'5'
DATA2 DC	F'10'		DATA2	DC	F'10'
.					
.					
.					

It is possible to supply more than one argument in a macro call. Each argument must correspond to a definition ("dummy") argument on the macro name line of the macro definition. When a macro call is processed, the arguments supplied are substituted for the respective dummy arguments in the macro definition.

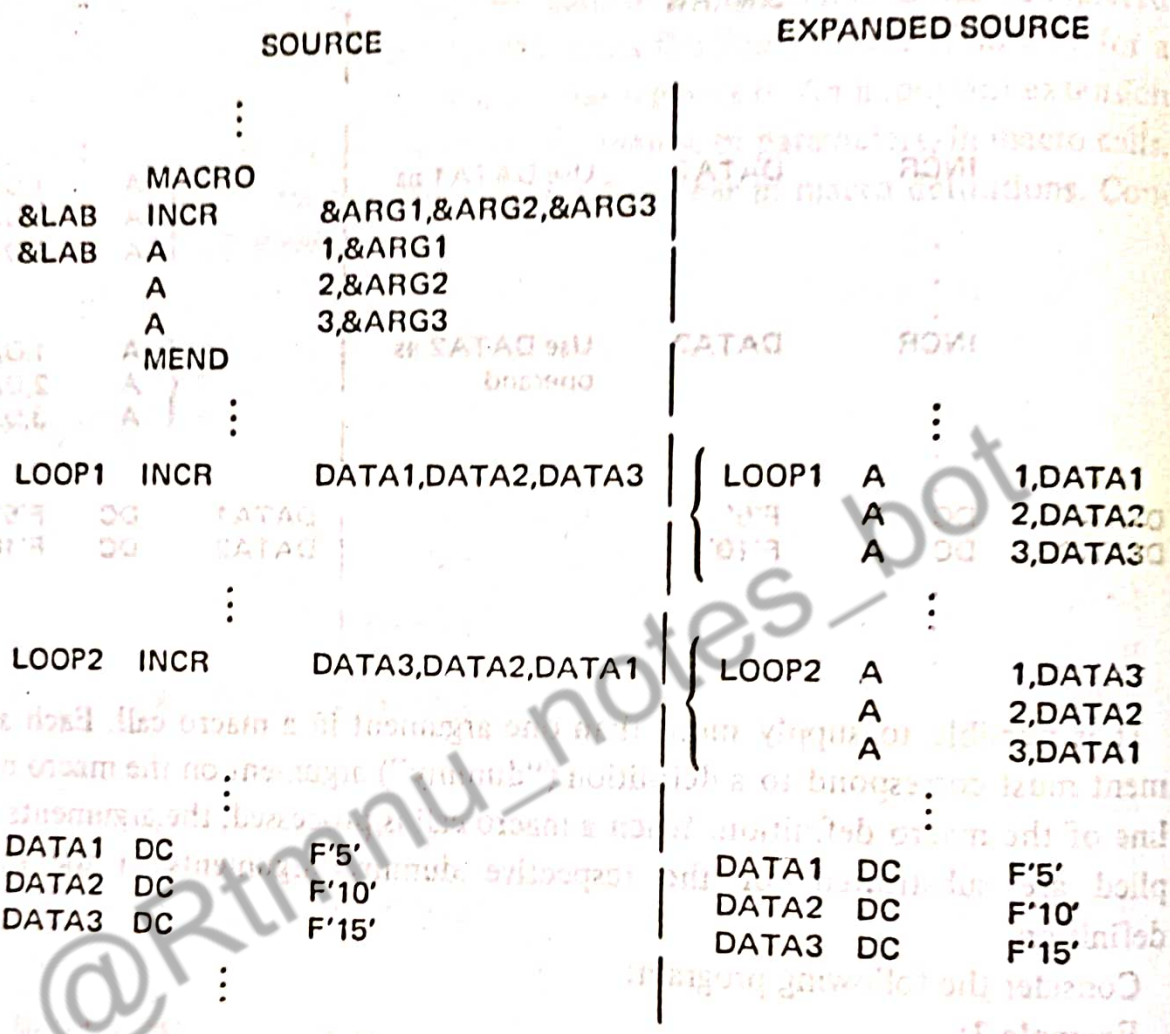
Consider the following program:

Example 3:

```
          .
          .
          .
LOOP1     A       1, DATA1
          A       2, DATA2
          A       3, DATA3
          .
          .
          .
LOOP2     A       1, DATA3
          A       2, DATA2
          A       3, DATA1
          .
          .
          .
DATA1     DC      F'5'
DATA2     DC      F'10'
DATA3     DC      F'15'
```

@Rmmu_notes_bot

In this case the operands in the common sequence are different, as are the labels on the first cards of the sequences. This program could be written as:



@Rtmnu_notes_bot

Here we have specified four arguments, including a *label argument.* Label arguments are treated just like those written as "operands" of a macro instruction; the argument appearing in the label field on the macro name line and in calls to the macro differs from other arguments only in its location. Arguments used as labels within the macro need not appear in the label field of a call, and arguments appearing in the label field of a call may be used as operands within the macro instruction code. Label arguments are convenient but nonessential features that enhance the similarity between macro instructions and ordinary assembly language instructions. Following is another encoding of example 3.

Source			Expanded source		
	MACRO				
	INCR	&ARG1,&ARG2,&ARG3,&LAB			
&LAB	A	1,&ARG1			
	A	2,&ARG2			
	A	3,&ARG3			
	MEND				
	.				
	.				
	.				
	INCR	DATA1,DATA2,DATA3,LOOP1	LOOP1	A	1,DATA1
	.			A	2,DATA2
	.			A	3,DATA3
	.				
	INCR	DATA3,DATA2,DATA1,LOOP2	LOOP2	A	1,DATA3
	.			A	2,DATA2
	.			A	3,DATA1

There are generally two ways of specifying arguments to a macro call. The first, *positional argument,* is demonstrated in the preceding examples. Arguments are matched with dummy arguments according to the order in which they appear. Thus, in a macro call 'INCR A,B,C' A, B, and C replace the first, second, and third dummy arguments. A more general argument facility, *keyword arguments*, allows reference to dummy arguments by name as well as by position. Referring to the dummy arguments appearing in the definition of INCR, we might write

'INCR &ARG1=A,&ARG3=C,&ARG2=B' or 'INCR &ARG1=A,&ARG2=&ARG3=C'.

This can be useful when a macro has arguments—labels, for example—that are not always needed. Any arguments not supplied are presumed blank by the macro processor.

4.2.2 Conditional Macro Expansion

Two important macro processor pseudo-ops, AIF and AGO, permit conditional reordering of the sequence of macro expansion. This allows conditional selection of the machine instructions that appear in expansions of a macro call.

Consider the following program:

@Ptmnu_notes_bot

Example 4:

```
          .
          .
          .
LOOP1     A       1,DATA1
          A       2,DATA2
          A       3,DATA3
          .
          .
          .
LOOP2     A       1,DATA3
          A       2,DATA2
          .
          .
          .
LOOP3     A       1,DATA1
          .
          .
          .
DATA1     DC      F'5'
DATA2     DC      F'10'
DATA3     DC      F'15'
          .
          .
          .
```

In this example, the operands, labels, and the number of instructions generated change in each sequence. This program could be written as follows:

```
          .
          .
          .
          MACRO
&ARG0     VARY    &COUNT,&ARG1,&ARG2,&ARG3
&ARG0     A       1,&ARG1
          AIF     (&COUNT EQ 1).FINI          Test if & COUNT = 1
          A       2,&ARG2
          AIF     (&COUNT EQ 2).FINI          Test if & COUNT = 2
          A       3,&ARG3
.FINI     MEND
          .
          .
          .
LOOP1     VARY    3,DATA1,DATA2,DATA3
          .
          .
          .
LOOP2     VARY    2,DATA3,DATA2
          .
          .
          .
LOOP3     VARY    1,DATA1
          .
          .
          .
DATA1     DC      F'5'
DATA2     DC      F'10'
DATA3     DC      F'15'
```

Expanded source

```
          .
          .
          .
LOOP1     A       1,DATA1
          A       2,DATA2
          A       3,DATA3
          .
          .
          .
LOOP2     A       1,DATA3
          A       2,DATA2
          .
          .
          .
LOOP3     A       1,DATA1
          .
          .
          .
```

@Rtnu_notes_bot

Labels starting with a period (.), such as .FINI, are *macro labels* and do not appear in the output of the macro processor. The statement AIF (&COUNT EQ 1)FINI directs the macro processor to skip to the statement labelled .FINI it the parameter corresponding to &COUNT is a 1; otherwise, the macro processor is to continue with the statement following the AIF pseudo-op.

AIF is a *conditional branch* pseudo-op; it performs an arithmetic test and branches only if the tested condition is true. The AGO is an *unconditional branch* pseudo-op or 'go to' statement. It specifies a label appearing on some other statement in the macro instruction definition; the macro processor continues sequential processing of instructions with the indicated statement. These statements are directives to the macro processor and do not appear in macro expansions. Just as conditional and unconditional branch instructions in a machine language program direct the order of program flow, AIF and AGO control the sequence in which the macro processor expands the statements in macro instructions. This makes it possible to tailor specific occurrences of instructions sequences to their various contexts. If machine language branches and tests are used, they will waste execution time; also, the tests and unused instructions will waste memory space. Branches and tests in the macro instruction language permit the use of highly general macros that assemble selectively, and by testing the parameters to each call, omit code not needed. This facility for selective assembly of highly general macros is one of the systems programmer's most powerful programming tools.

@Rtmnu_notes_bot

4.2.3 Macro Calls Within Macros

Since macro calls are "abbreviations" of instruction sequences, it seems reasonable that such "abbreviations" should be available within other macro definitions. For example,

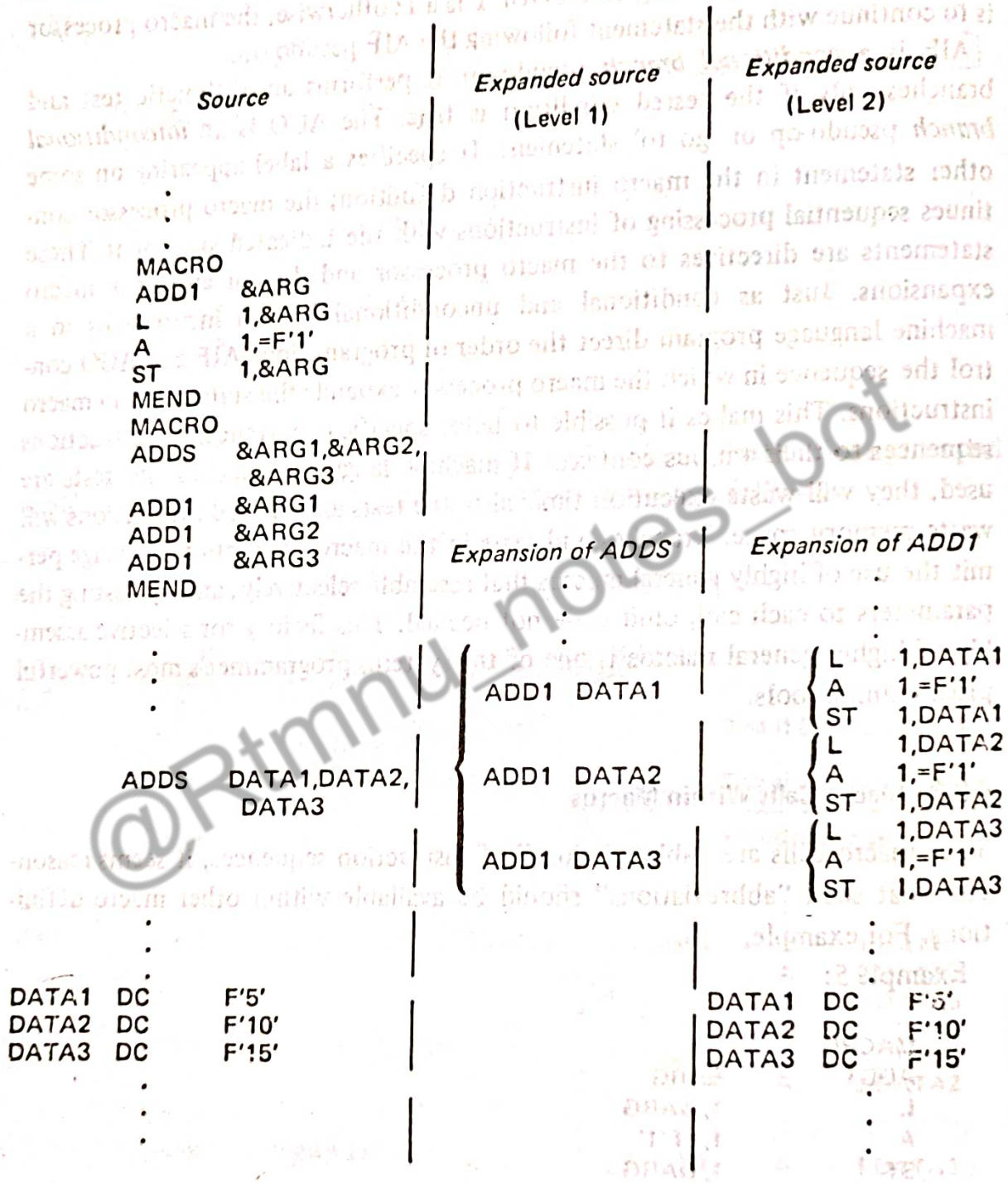
Example 5:

```
MACRO
ADD1    &ARG
L       1,&ARG
A       1,=F'1'
ST      1,&ARG
MEND
MACRO
ADDS    &ARG1,&ARG2,&ARG3
ADD1    &ARG1
ADD1    &ARG2
ADD1    &ARG3
MEND
```

Within the definition of the macro 'ADDS' are three separate calls to a pre-

viously defined macro 'ADD1'. The use of the macro 'ADD1' has shortened the length of the definition of 'ADDS' and thus has made it more easily understood.

Such use of macros results in macro expansions on multiple 'levels'. Thus:



Macro calls within macros can involve several levels. For example, the macro ADDS might be called within the definition of another macro. In fact, conditional macro facilities (such as AIF and AGO) make it possible for a macro to call itself. So long as this does not cause an infinite loop—so long as at some point the macro, having been called for the nth time, decides not to call itself

@Rtmnu_notes_bot

again—it makes perfectly good sense. The problems inherent to macro calls within macros will be discussed in the section on *recursion*; the preceding example is treated further in Figure 4.5.

4.2.4 Macro Instructions Defining Macros

We have viewed macros as generalized abbreviations for instruction sequences, noting that it seems reasonable to permit any valid statements in the abbreviated sequence, including macro definitions. In this manner a single macro instruction might be used to simplify the process of defining a group of similar macros.

It is important to realize that the inner macro definition is not defined (i.e., callable) until after the outer macro has been called. This is because of the method by which definitions are implemented. For example, a user might wish to define a group of macros for subroutine calls with some standardized calling sequence. The following example defines a macro instruction DEFINE, which when called with a subroutine name defines a macro with the same name as the subroutine. The individual macros generated bear the names (given through the argument &SUB) of their associated subroutines.

```
Definition of macro DEFINE:
        MACRO
        DEFINE  &SUB                    Macro name: DEFINE
        Definition of macro &SUB:
                MACRO
                &SUB    &Y              Dummy macro name
                CNOP    0,4             Align boundary
                BAL     1,*+8           Set register 1 to parameter list pointer
                DC      A(&Y)           Parameter list pointer
                L       15,=V(&SUB)     Address of subroutine
                BALR    14,15           Transfer control to subroutine
                MEND
        MEND
```

The user might then call this macro with the statement

```
DEFINE      COS
```

defining a new macro named COS: the statement expands into a new macro definition. The user might subsequently call the COS macro as follows:

```
COS         AR
```

and the macro processor will generate the calling sequence:

@ptunu_notes_bot

```
BAL     1,*+8
DC      A(AR)          address of AR
L       15,=V(COS)     V denotes address of external symbol
BALR    14,15
```

Macro definitions within macros are sometimes called "macro definitions within macro definitions." (See Section 4.3.2 for further treatment of this example.)

4.3 IMPLEMENTATION

Thus far we have described a macro instruction scheme for extending the basic assembly language. In the remainder of this chapter we will outline a method for implementing such a scheme.

STATEMENT OF PROBLEM

There are four basic tasks that any macro instruction processor must perform.

1. *Recognize macro definitions* A macro instruction processor must recognize macro definitions identified by the MACRO and MEND pseudo-ops. This task can be complicated when macro definitions appear within macros. When MACROs and MENDs are *nested*, as in the example of the previous section, the macro processor must recognize the nesting and correctly match the last or outer MEND with the first MACRO. All of the intervening text, including nested MACROs and MENDs, defines a single macro instruction.

2. *Save the definitions* The processor must store the macro instruction definitions, which it will need for expanding macro calls.

3. *Recognize calls* The processor must recognize macro calls that appear as operation mnemonics. This suggests that macro names be handled as a type of op-code.

4. *Expand calls and substitute arguments* The processor must substitute for dummy or macro definition arguments the corresponding arguments from a macro call; the resulting symbolic (in this case, assembly language) text is then substituted for the macro call. This text, of course, may contain additional macro definitions or calls.

In summary: the macro processor must recognize and process macro definitions and macro calls.

With regard to arguments, the designer of the macro processor must make several decisions. He must determine where dummy arguments may appear in a macro definition – for example, whether they may appear as op-codes. He must

@Pimmu_notes_bot

also define the syntax of permissible arguments. Although we do not treat these matters in detail, we present some typical (and reasonable) answers. Dummy arguments may appear *anywhere* in a macro definition – for example, they may appear as op-codes. We want to be able to concatenate macro arguments with fixed strings. For example, we might want to write A&A, which we take to mean A concatenated with the argument replacing &A. This is easy to arrange. It is harder to have arguments inside a string. For this we permit the argument to be enclosed in single quotes. Thus, we take A&AB to mean A concatenated with the argument replacing AB, and A'&A'B to mean the concatenation of A, the argument replacing &A, and *B*. Any string enclosed in parentheses (e.g., (.A&B)) may be used as an argument.

In order for AIF and AGO to work, some arithmetic expressions will have to be evaluated. A symbol defining capability, similar to the EQU facility by which variables may be assigned and reassigned values, is often useful.

4.3.1 Implementation of a Restricted Facility: A Two-Pass Algorithm

We begin by making some simplifying assumptions. We will assume that our macro processor is functionally independent of the assembler and that the output text from the macro processor will be fed into the assembler. Initially we will not permit macro calls or definitions within macro definitions. Because these features introduce complications, we postpone discussion of them until after we have presented the basic macro processor algorithm.

A macro processor, like an assembler, scans and processes lines of a text. In assembly language, lines are interrelated by addressing: a line can refer to another by its address or name, which must be available ("known") to the assembler. Moreover, the address assigned to each line depends upon preceding lines, upon their addresses, and possibly upon their contents as well. If we consider macro definitions to constitute integral entities, we may say that the lines of our macro language are not so closely interrelated. Macro definitions refer to nothing outside themselves, and macro calls refer only to macro definitions. (Remember our restrictions. Remember also that a macro call substitutes *text*, not values, for parameters. Suppose, for example, that a macro call INCR Y is preceded by a statement defining Y–e.g., Y EQU 10. The macro processor substitutes 'Y', not '10' for the macro definition argument–indeed, it never processes the EQU statement at all, except as a complete line of text.)

Our macro processor algorithm will make two systematic scans, or passes, over the input text, searching first for macro definitions and then for macro calls. Just as our assembler cannot process a reference to a symbol before its definition, so the macro processor cannot expand a macro call before having found

@Pmnu_notes_bot

and saved the corresponding macro definition. Thus we need the two passes over the input text, one to handle definitions and one to handle calls. The first pass, examining every operation code, will save all macro definitions in a Macro Definition Table (MDT) and save a copy of the input text, minus macro definitions, on secondary storage (e.g., magnetic tape) for use in the second pass. The first pass will also prepare a Macro Name Table (MNT) along with the MDT. The second pass will then examine every operation mnemonic and replace each macro name with the appropriate text from the macro definitions. (A separate MNT is not essential, but it facilitates searching for macro names.)

SPECIFICATION OF DATA BASES

The following data bases are used by the two passes of the macro processor.

Pass 1 data bases:

1. The input macro source deck
2. The output macro source deck copy for use by pass 2
3. The Macro Definition Table (MDT), used to store the body of the macro definitions
4. The Macro Name Table (MNT), used to store the names of defined macros
5. The Macro Definition Table Counter (MDTC), used to indicate the next available entry in the MDT
6. The Macro Name Table Counter (MNTC), used to indicate the next available entry in the MNT
7. The Argument List Array (ALA), used to substitute index markers for dummy arguments before storing a macro definition

Pass 2 data bases:

1. The copy of the input macro source deck
2. The output expanded source deck to be used as input to the assembler
3. The Macro Definition Table (MDT), created by pass 1
4. The Macro Name Table (MNT), created by pass 1
5. The Macro Definition Table Pointer (MDTP), used to indicate the next line of text to be used during macro expansion
6. The Argument List Array (ALA), used to substitute macro call arguments for the index markers in the stored macro definition

SPECIFICATION OF DATA BASE FORMAT

The only data bases with nontrivial format are the Macro Definition Table (MDT), the Macro Name Table (MNT), and the Argument List Array (ALA). The others we will not present in detail.

ARGUMENT LIST ARRAY: The Argument List Array (ALA) is used during both pass 1 and pass 2 but for somewhat reverse functions. During pass 1, in order to simplify later argument replacement during macro expansion, dummy

©Rtanu_notes_bot

arguments in the macro definition are replaced with positional indicators when the definition is stored. The ith dummy argument on the macro name card is represented in the body of the macro by the index marker symbol # where #i is a symbol reserved for the use of the macro processor (i.e., not available to the programmers). These symbols are used in conjunction with the argument list prepared before expansion of a macro call. The symbolic dummy arguments are retained on the macro name card to enable the macro processor to handle argument replacement by name rather than by position.

As an example, consider the macro INCR used in Example 3. The stored macro definition would be:

Macro Definition Table MDT

```
          ⋮
&LAB      INCR    &ARG1,&ARG2,&ARG3
#0        A       1, #1
          A       2, #2
          A       3, #3
          MEND
          ⋮
```

During pass 2 it is necessary to substitute macro call arguments for the index markers stored in the macro definition. Thus upon encountering the call

```
LOOP      INCR    DATA1,DATA2,DATA3
```

the macro call expander would prepare an argument list array:

Argument List Array

8 bytes per entry

Index	*Argument*
0	"LOOP1*bbb*"
1	"DATA1*bbb*"
2	"DATA2*bbb*"
3	"DATA3*bbb*"

(*b* denotes the blank character)

The list would be used only while expanding this particular call. Suppose that a succeeding call were:

```
INCR      &ARG1=DATA3,&ARG2=DATA2,&ARG3=DATA1
```

@Rtmnu_notes_bot

The macro processor would find that '&ARG1', '&ARG2', and '&ARG3' occupy argument positions 1, 2, and 3 on the macro name card. The resulting argument list array would be:

Argument List Array

Index	*Argument*
0	*"bbbbbbbb"* (all blank)
1	"DATA3*bbb*"
2	"DATA2*bbb*"
3	"DATA1*bbb*"

MACRO DEFINITION TABLE: The Macro Definition Table (MDT) is a table of text lines; if input is from 80-column cards, the MDT can be a table with 80-byte strings as entries. Every line of each macro definition, except the MACRO line, is stored in the MDT. (The MACRO line is useless during macro expansion.) The MEND is kept to indicate the end of the definition; and the macro name line is retained to facilitate keyword argument replacement. Thus, for example, the INCR macro discussed might be stored as follows:

Macro Definition Table

80 bytes per entry

Index	*Card*		
⋮		⋮	
15	&LAB	INCR	&ARG1,&ARG2,&ARG3
16	#0	A	1,#1
17		A	2,#2
18		A	3,#3
19		MEND	
⋮		⋮	

MACRO NAME TABLE: The Macro Name Table (MNT) serves a function very similar to that of the assembler's Machine-Op Table (MOT) and Pseudo-Op Table (POT). Each MNT entry consists of a character string (the macro name) and a pointer (index) to the entry in the MDT that corresponds to the beginning of the macro definition. The MNT entry for the INCR macro discussed might be:

Index	8 bytes *Name*	4 bytes *MDT index*
⋮	⋮	⋮
3	"INCR*bbbb*"	15
⋮	⋮	⋮

@Rtmnu_notes_bot

ALGORITHM

Figures 4.1 and 4.2 are flowcharts of the macro definition and expansion algorithms. Each of the algorithms makes a line-by-line scan over its input. The 'READ' boxes refer to the fetching of successive input lines from secondary storage into a workspace.

PASS 1–MACRO DEFINITION: The algorithm for pass 1 (Fig. 4.1) tests each input line. If it is a MACRO pseudo-op, the entire macro definition that follows is saved in the next available locations in the Macro Definition Table (MDT). The first line of the definition is the macro name line. The name is entered into the Macro Name Table (MNT), along with a pointer to the first location of the MDT entry of the definition. When the END pseudo-op is encountered, all of the macro definitions have been processed so control transfers to pass 2 in order to process macro calls.

PASS 2–MACRO CALLS AND EXPANSION: The algorithm for pass 2 (Fig. 4.2) tests the operation mnemonic of each input line to see if it is a name in the MNT. When a call is found, the call processor sets a pointer, the Macro Definition Table Pointer (MDTP), to the corresponding macro definition stored in the MDT. The initial value of the MDTP is obtained from the "MDT index" field of the MNT entry. The macro expander prepares the Argument List Array (ALA) consisting of a table of dummy argument indices and corresponding arguments to the call. This list is simply a succession of symbols ordered to match the dummy arguments on the name card (the first is the label argument, which is considered to have an index of zero). Arguments not represented in a call are considered blank, and superfluous arguments are ignored. In the case of argument reference by position, this scheme is completely straightforward. For references by name, the macro processor locates the dummy argument on the macro name line (which is available at the beginning of the definition in the MDT) in order to determine the proper index.

Reading proceeds from the MDT; as each successive line is read, the values from the argument list are substituted for dummy argument indices in the macro definition. Reading of the MEND line in the MDT terminates expansion of the macro, and scanning continues from the input file. When the END pseudo-op is encountered, the expanded source deck is transferred to the assembler for further processing.

@Rimnl_notes_bot

4.3.2 A Single-Pass Algorithm

Suppose we wanted to provide for macro definitions within macros. The basic problem here is that the inner macro is defined only after the outer one has been called; in order to provide for any use of the inner macro, we would have to re-

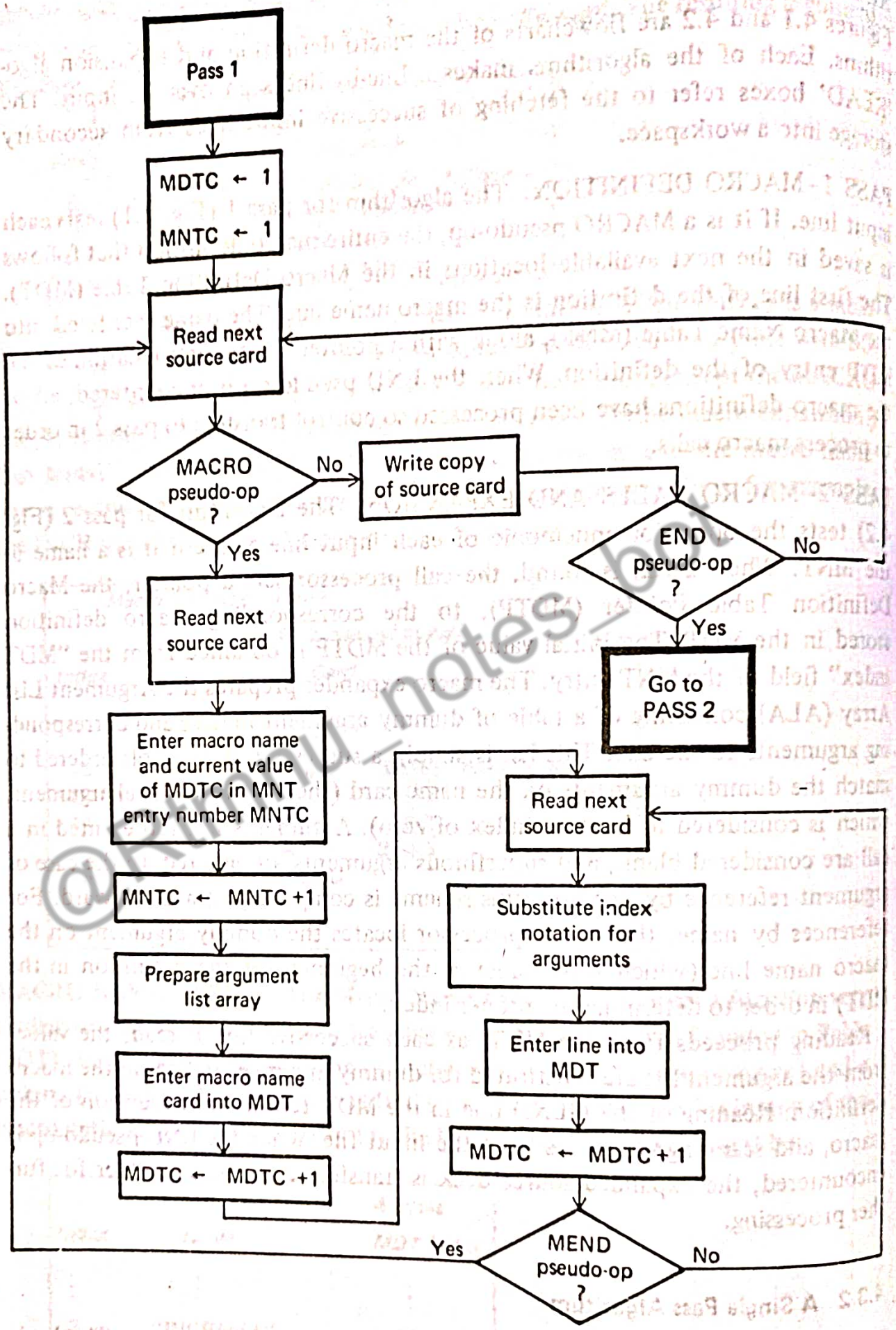
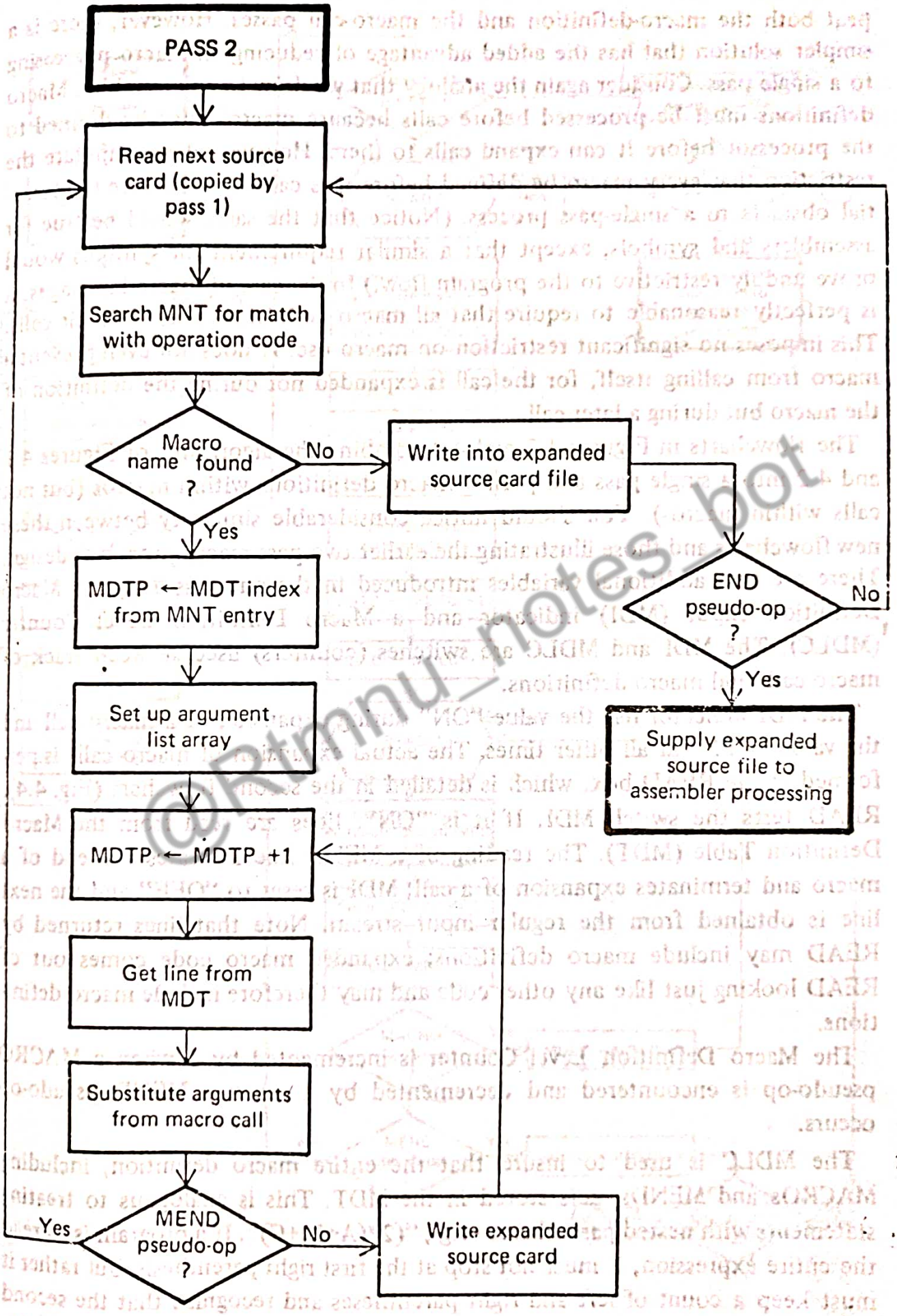


FIGURE 4.1 Pass 1–processing macro definitions

@Rimu_notes_bot



@Rtmnu_notes_bot

FIGURE 4.2 Pass 2–processing macro calls and expansion

peat both the macro-definition and the macro-call passes. However, there is a simpler solution that has the added advantage of reducing all macro processing to a single pass. Consider again the analogy that we drew to an assembler. Macro definitions must be processed before calls because macros must be defined to the processor before it can expand calls to them. However, if we stipulate the restriction that every macro be defined before it is called, we remove the essential obstacle to a single-pass process. (Notice that the same would be true for assemblers and symbols, except that a similar requirement for symbols would prove unduly restrictive to the program flow.) In the case of macro languages, it is perfectly reasonable to require that all macro definitions precede their calls. This imposes no significant restriction on macro use. It does not even prevent a macro from calling itself, for the call is expanded not during the definition of the macro but during a later call.

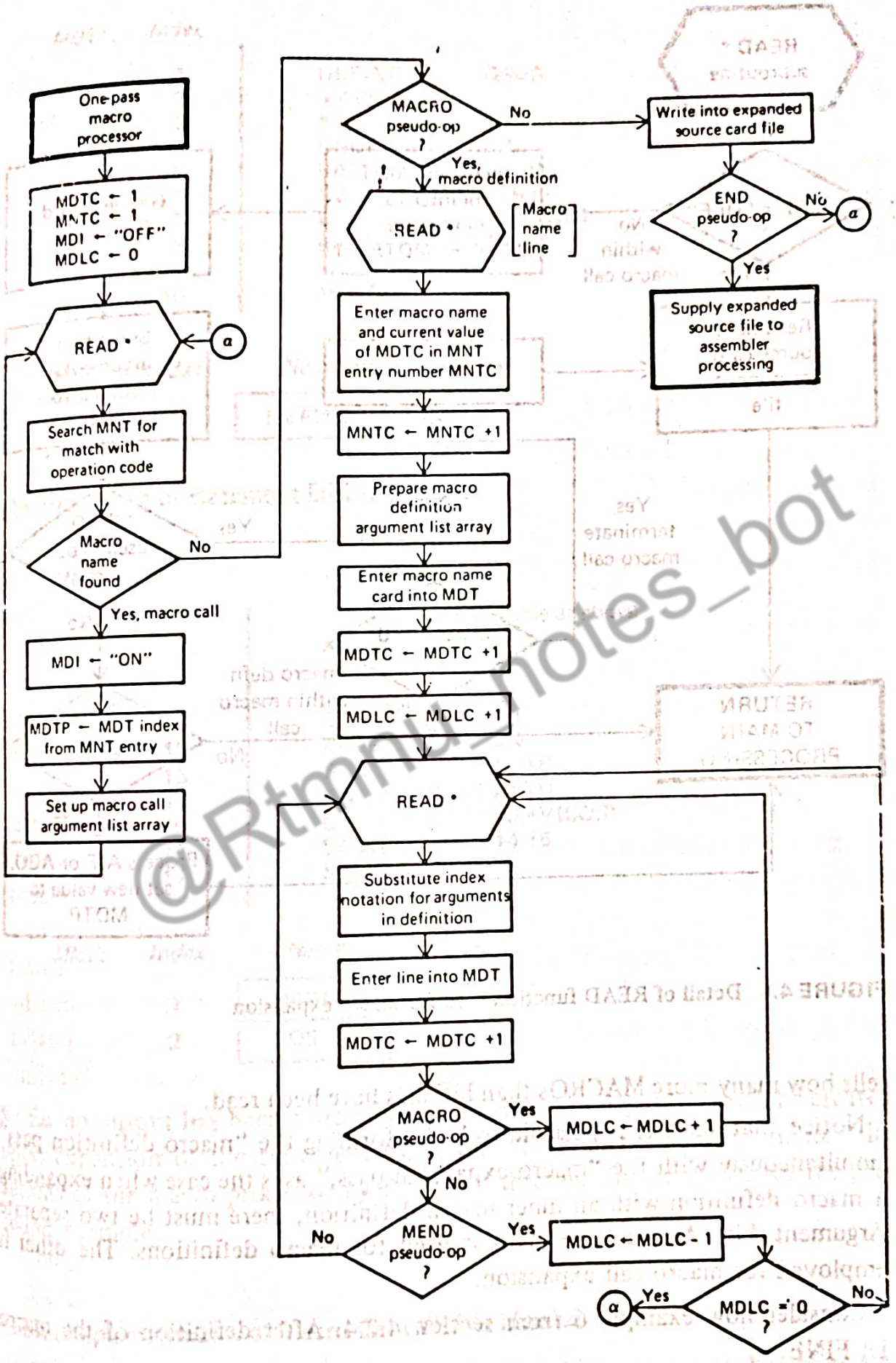
The flowcharts in Figures 4.3 and 4.4 combine the algorithms of Figures 4.1 and 4.2 into a single pass and permit macro definitions within macros (but not calls within macros). You should notice considerable similarity between these new flowcharts and those illustrating the earlier two-pass macro processor design. There are two additional variables introduced in the one-pass design: a Macro Definition Input (MDI) indicator and a Macro Definition Level Counter (MDLC). The MDI and MDLC are switches (counters) used to keep track of macro calls and macro definitions.

The MDI indicator has the value "ON" during expansion of a macro call and the value "OFF" at all other times. The actual expansion of macro calls is performed in the READ box, which is detailed in the second flowchart (Fig. 4.4). READ tests the switch MDI. If it is "ON", lines are read from the Macro Definition Table (MDT). The reading of a MEND line indicates the end of a macro and terminates expansion of a call; MDI is reset to "OFF" and the next line is obtained from the regular input stream. Note that lines returned by READ may include macro definitions; expanded macro code comes out of READ looking just like any other code and may therefore include macro definitions.

The Macro Definition Level Counter is incremented by 1 when a MACRO pseudo-op is encountered and decremented by 1 when a MEND pseudo-op occurs.

The MDLC is used to insure that the entire macro definition, including MACROs and MENDs, gets stored in the MDT. This is analogous to treating statements with nested parentheses—e.g., "(2*(A+B)+C)". If a program is to read the entire expression, it must not stop at the first right parenthesis, but rather it must keep a count of left and right parentheses and recognize that the second parenthesis is the true end of the expression. The MDLC acts as a counter for the difference between the numbers of MACROs and MENDs encountered; it

@Rmmu_notes_bot



@Rtmnu_notes_bot

FIGURE 4.3 Simple one-pass macro processor

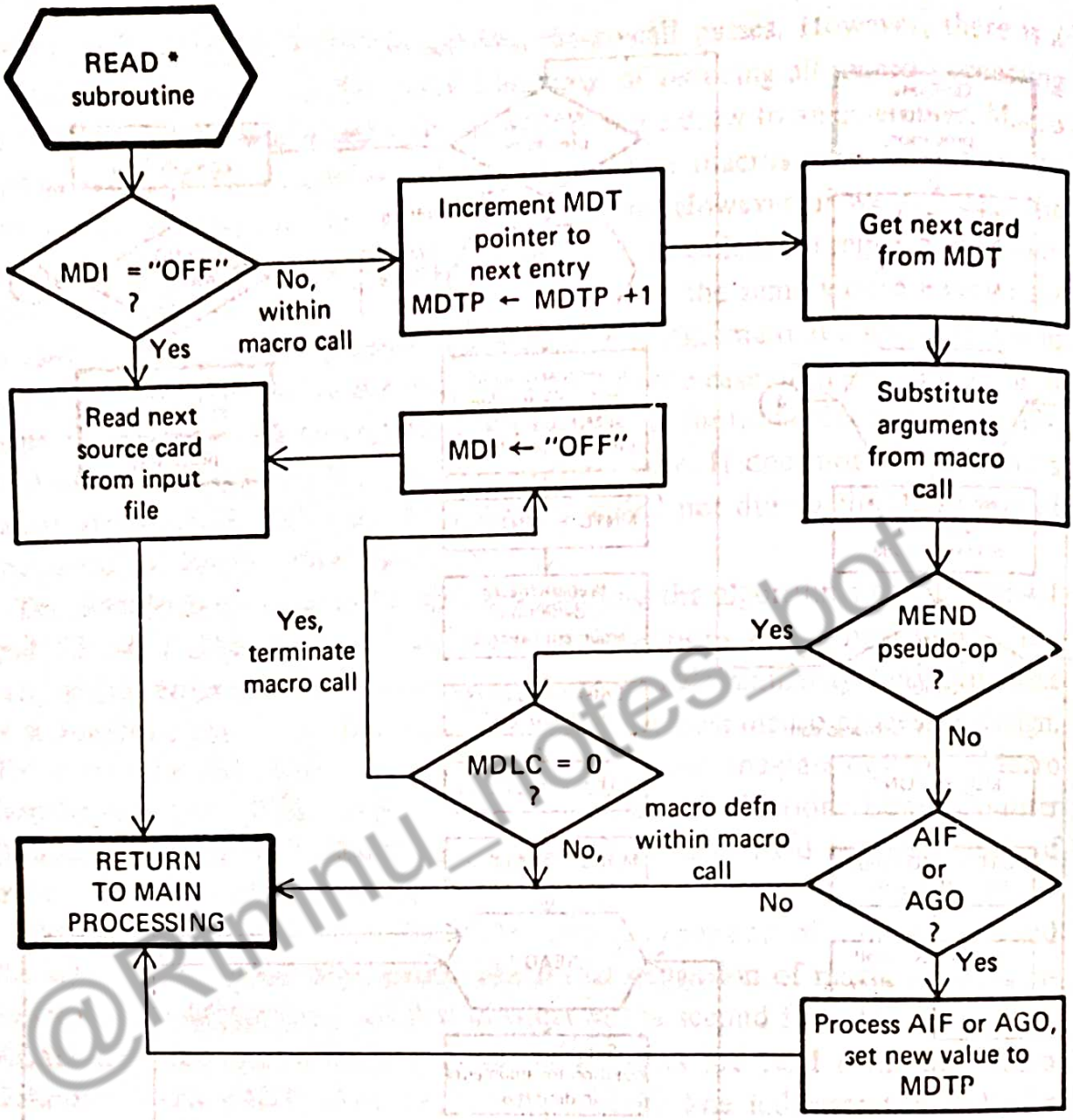


FIGURE 4.4 Detail of READ function used for macro expansion

tells how many more MACROs than MENDs have been read.

Notice that since it is possible to be performing the "macro definition pass" simultaneously with the "macro expansion pass," as is the case when expanding a macro definition with an inner macro definition, there must be two separate Argument List Arrays. One ALA is used for macro definitions. The other is employed for macro call expansion.

Consider now example 6 from section 4.2.4. After definition of the macro DEFINE:

MDT:

Index			
1		DEFINE	&SUB
2		MACRO	
3		#1	&Y
4		CNOP	0,4
5		BAL	1,*+8
6		DC	A(&Y)
7		L	15,=V(#1)
8		BALR	14,15
9		MEND	
10		MEND	

MNT:

Index	Name	MDT Index
1	DEFINE	1

after processing of statement DEFINE COS:

MDT:

		Lines 1-10 same as above	
.		.	
.		.	
.		.	
11		COS	&Y
12		CNOP	0,4
13		BAL	1,*+8
14		DC	A(#1)
15		L	15,=V(COS)
16		BALR	14,15
17		MEND	

MNT:

Index	Name	MDT Index
1	DEFINE	1
2	COS	11

So far no output has been generated by the macro: output lines will be created upon expansion of the statement COS AR. The reader should "simulate" the actions of the macro processor by following through the flowchart for the preceding example.

4.3.3 Implementation of Macro Calls Within Macros

The basic problem in implementing macro calls within macros is that of *recur-*

@Rtmnu_notes_bot

sion. If a macro call is encountered during the expansion of a macro, the macro processor will have to expand the included macro call and then finish expanding the enclosing macro. The second call might be expanded by a second macro processor, which would look up the macro definition in the MDT and return the expanded code to the first macro processor. Having many macro processors is neither efficient nor general; however, if a single macro processor is to handle such nested macro calls, it must in some way save its status when it encounters nested calls.

Example 5 illustrates this problem. It defines two macros, ADDS and ADD1. The macro definitions in the MDT are shown in Figure 4.5.

Macro Definition Table MDT

Index	*Contents*	
1	ADD1	&ARG
2	L	1,#1
3	A	1,=F'1'
4	ST	1,#1
5	MEND	
6	ADDS	&ARG1,&ARG2,&ARG3
7	ADD1	#1
8	ADD1	#2
9	ADD1	#3
10	MEND	

FIGURE 4.5 MDT after macro definition of example 5

Consider the action of the macro processor of Figure 4.3 and 4.4 when it encounters the macro call

```
ADDS        DATA1, DATA2, DATA3
```

Our algorithm will prepare a macro call argument list array and set a pointer, MDTP, to line 6 of the MDT; switch MDI is set to "ON". The READ function increments MDTP, gets its next line from the MDT (line 7) and substitutes the argument, yielding the line

```
ADD1        DATA1
```

Now our algorithm is in trouble: it has encountered another macro call. ADD1 is a macro name, so the processor will prepare a new argument list array, set MDTP to line 1 of the MDT, and set MDI to "ON", its current value. The ADD1 macro will expand correctly; at the end MDTP will point to MDT line 5. MDI will be re-

@Rtmnu_notes_bot

set to "OFF", and reading will proceed from the input stream.

This of course is wrong: there are unexpanded lines left in the ADDS macro. Three errors have occurred. The switch MDI has been turned off, causing the processor to read from the input stream instead of from the MDT; the value of MDTP before it was reset to process the ADD1 macro has been lost; and the argument list for the call to ADDS has been lost.

To handle macro calls within macros, the macro processor must be able to work recursively—that is, to process one macro before it is finished with another, then to continue with the previous, or 'outer,' one. Recursive procedures usually operate by means of a *stack*, a storage scheme that allocates a separate storage area for the variables associated with each call to the procedure. Because a separate storage space or 'stack frame' is associated with each recursive call, the status of unfinished computations is preserved. Conceptually, a stack is an unbounded array that is treated in a last-in, first-out manner: the last element stored is the first one removed. A Stack Pointer (SP) indicates the position or *frame* at the 'top' of the stack. *(Pointer* is a term that refers to a data item which is an address or index. Intuitively, a pointer points to information stored in core.)

In our example the pointer MDTP and the argument list ALA associated with each call belong in the stack. The switch MDI must also be saved. The Stack Pointer (SP), which indicates the beginning of the current stack frame, can serve in place of MDI; in each succeeding frame, however, we must save the previous value of SP to prevent it from being lost. (Compare this with our use of the Macro Definition Level Counter (MDLC). 'Recursive' handling of macro definitions was easy because the inner definitions do not have any associated data—see the exercises at the end of this chapter). Ignoring the actual method of data representation, we consider the stack to be an array of pointers and character strings; S(SP) refers to the beginning of the current (top) stack frame, the position indexed by SP.

The organization of each stack frame is depicted in Figure 4.6. S(SP) contains the *previous* value of SP; for the first frame, where SP=1,S(SP)=-1. The condition SP=-1, like MDI=0 in the nonrecursive algorithm, tells the macro processor that it is not within a macro call expansion. S(SP+1) contains the current value of MDTP, the MDT pointer; S(SP+2) . . . S(SP+N+1) contain the N (0th through (N-1)th) character strings of the current argument list.

Figures 4.7 and 4.8 depict the macro processor algorithm that handles macro calls within macro definitions as well as macro definitions within macro definitions. Simple "housekeeping" steps, such as incrementing the Macro Name Table Counter (MNTC) and the Macro Definition Table Counter (MDTC) after inserting a new entry, have been omitted to simplify the flowcharts. The reader

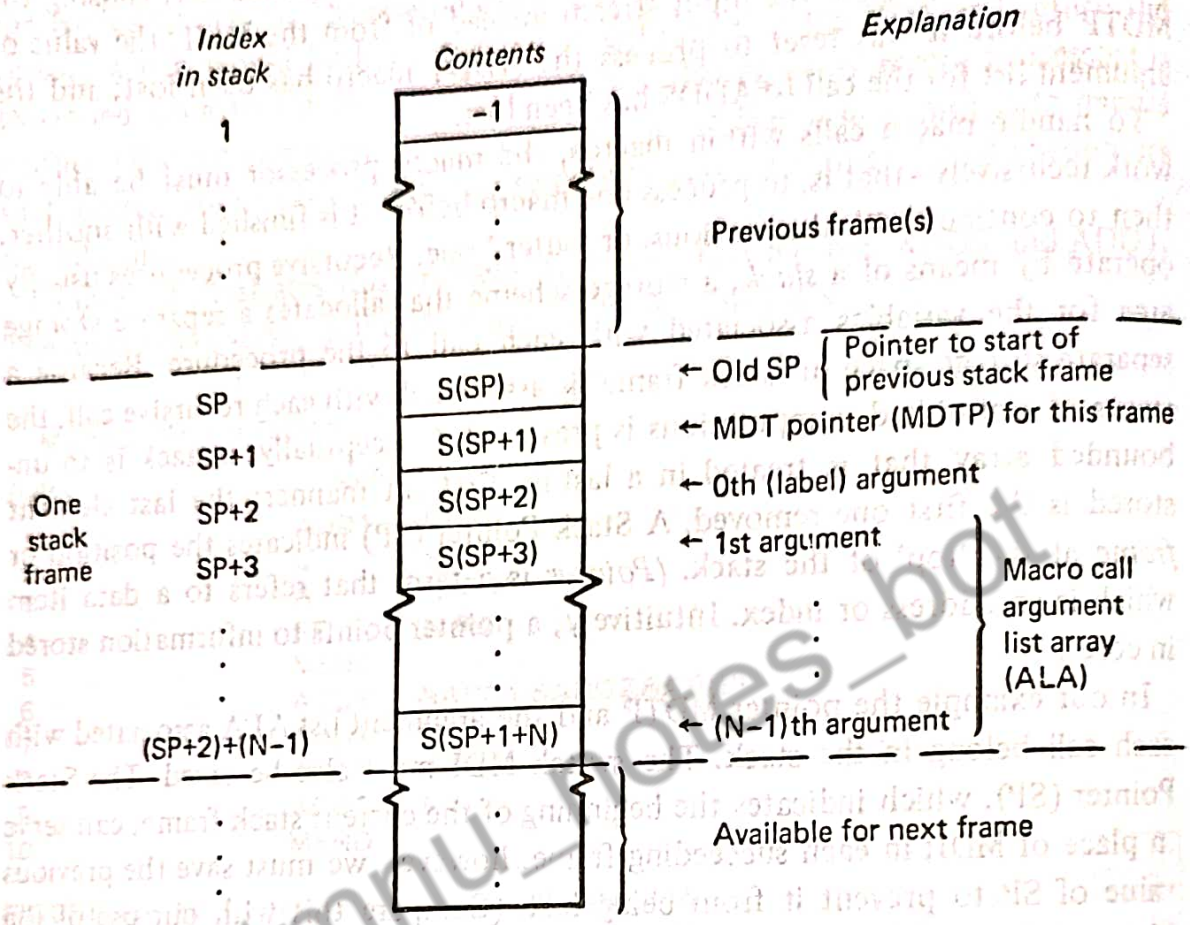


FIGURE 4.6 Stack organization

should confirm the fact that the algorithm described in these new flowcharts is indeed very similar to the previous algorithm (Figs. 4.3 and 4.4). Notice that the number of entries in a stack frame depends upon the number of elements in the argument list (i.e., the number of dummy arguments in the macro definition).

It is desirable for the reader to hand-simulate the operation of the macro processor by using the algorithm to expand a macro source program. Figure 4.9 illustrates the state of the stack and expanded source at various key points in the processing of Example 5. The corresponding Macro Definition Table (MDT) for this example was presented in Figure 4.5. It is strongly recommended that the reader go through all the steps of the macro expansion and compare results with Figures 4.9a and 4.9b.

@Ptmnu_notes_bot

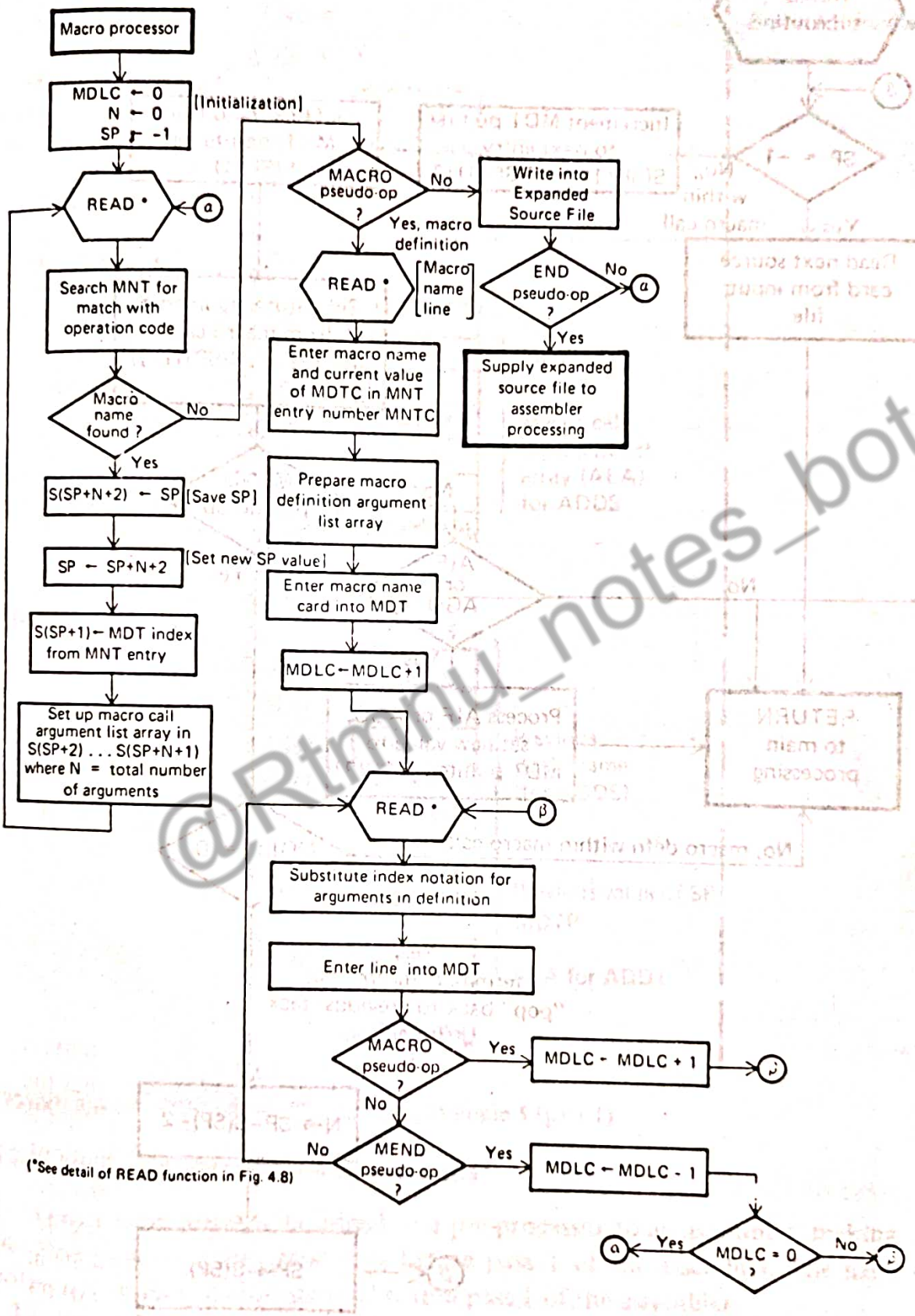


FIGURE 4.7 One-pass macro processor capable of handling macro calls within macro definitions

@Rtmnu_notes_bot

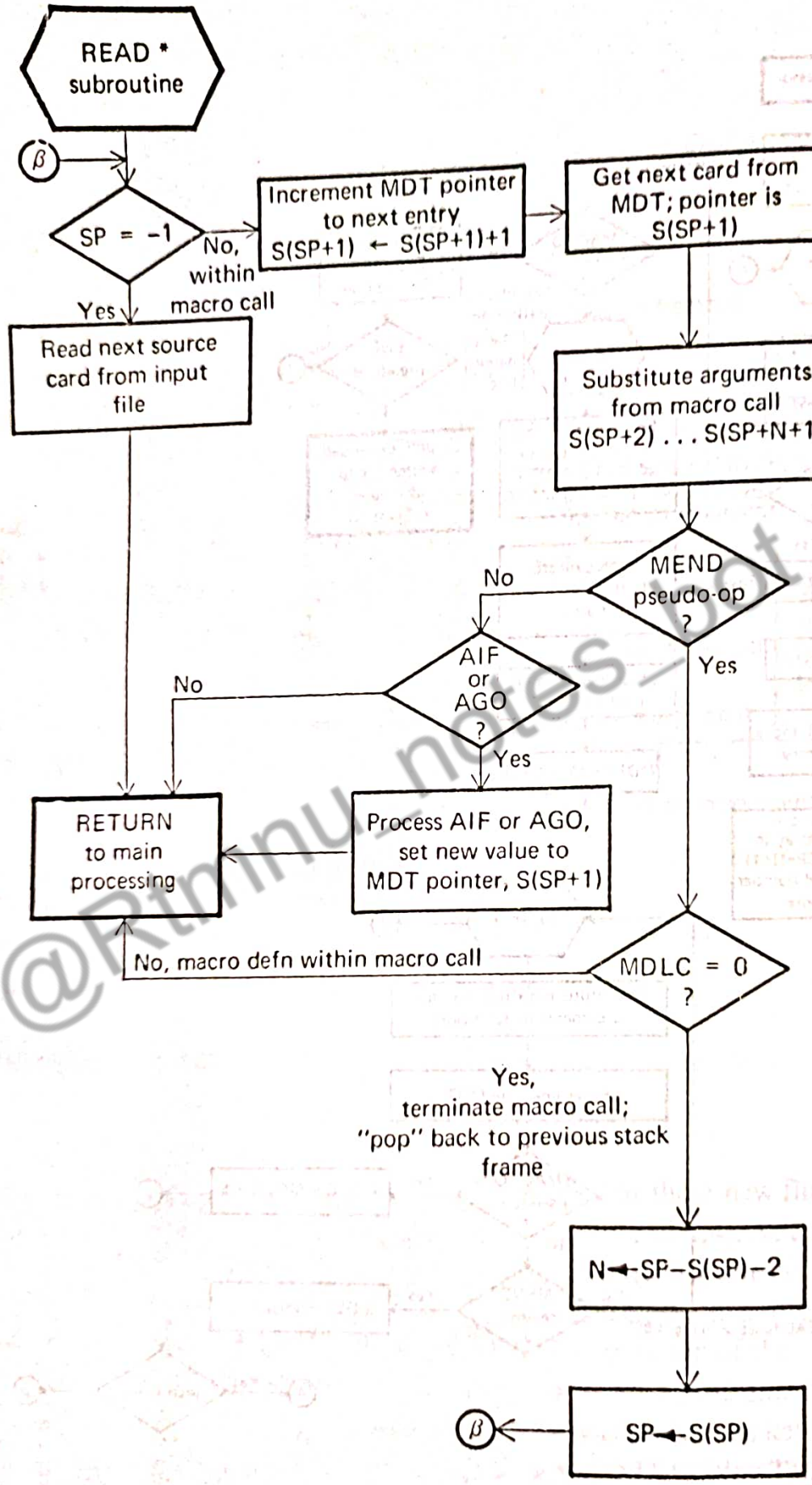
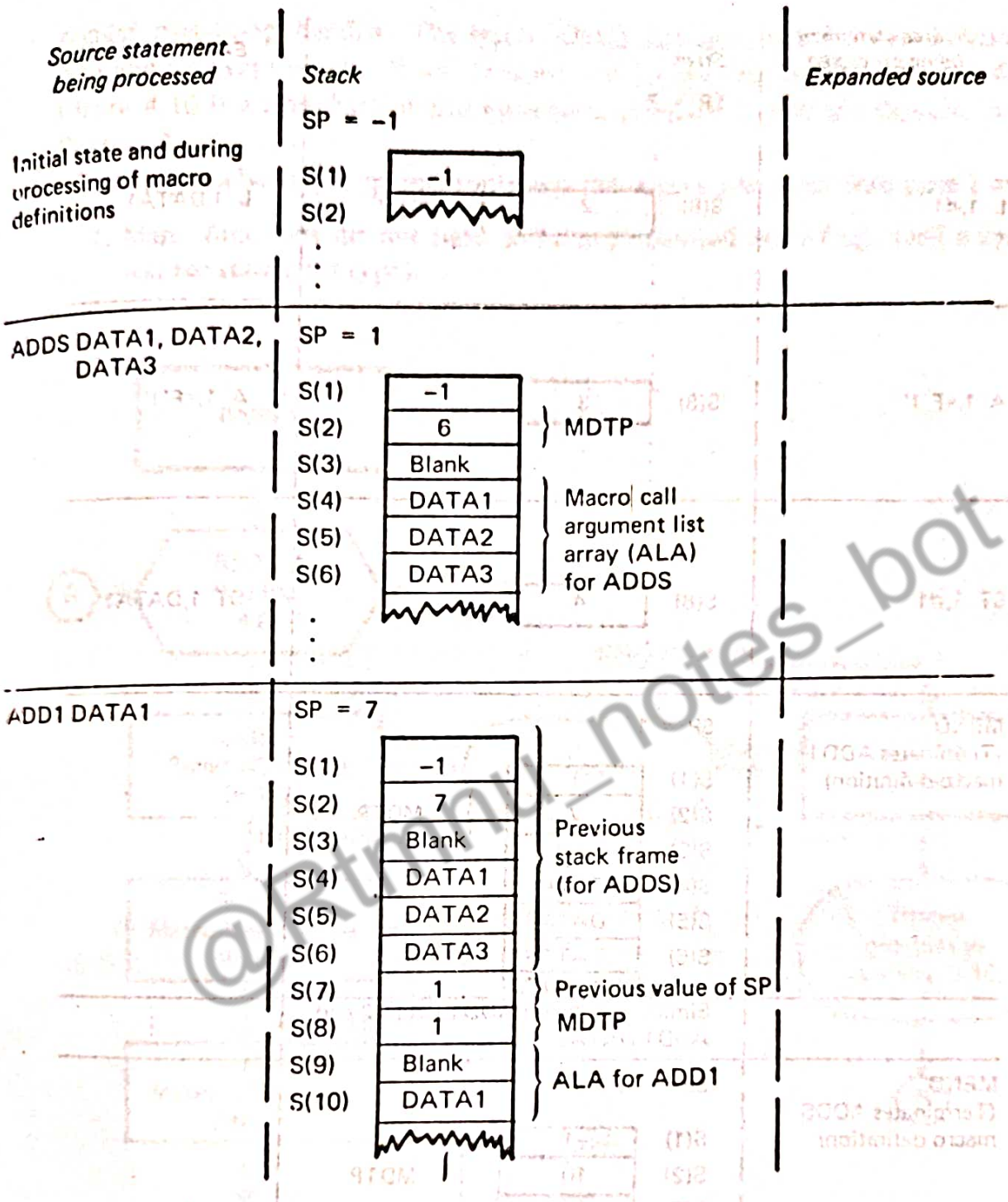


FIGURE 4.8 Detail of READ function for recursive macro expansion

@Rtmnu_notes_bot



@rtmnu_notes_bot

FIGURE 4.9 (a) Chronology of processing example 5 (part 1)

4.3.4 Implementation Within an Assembler

The macro processor can be added as a pre-processor to an assembler, making a complete pass over the input text before pass 1 of the assembler. The macro processor can also be implemented within pass 1 of the assembler.

The implementation of the macro processor within pass 1 eliminates the overhead of intermediate files, and we can improve this integration of macro pro-

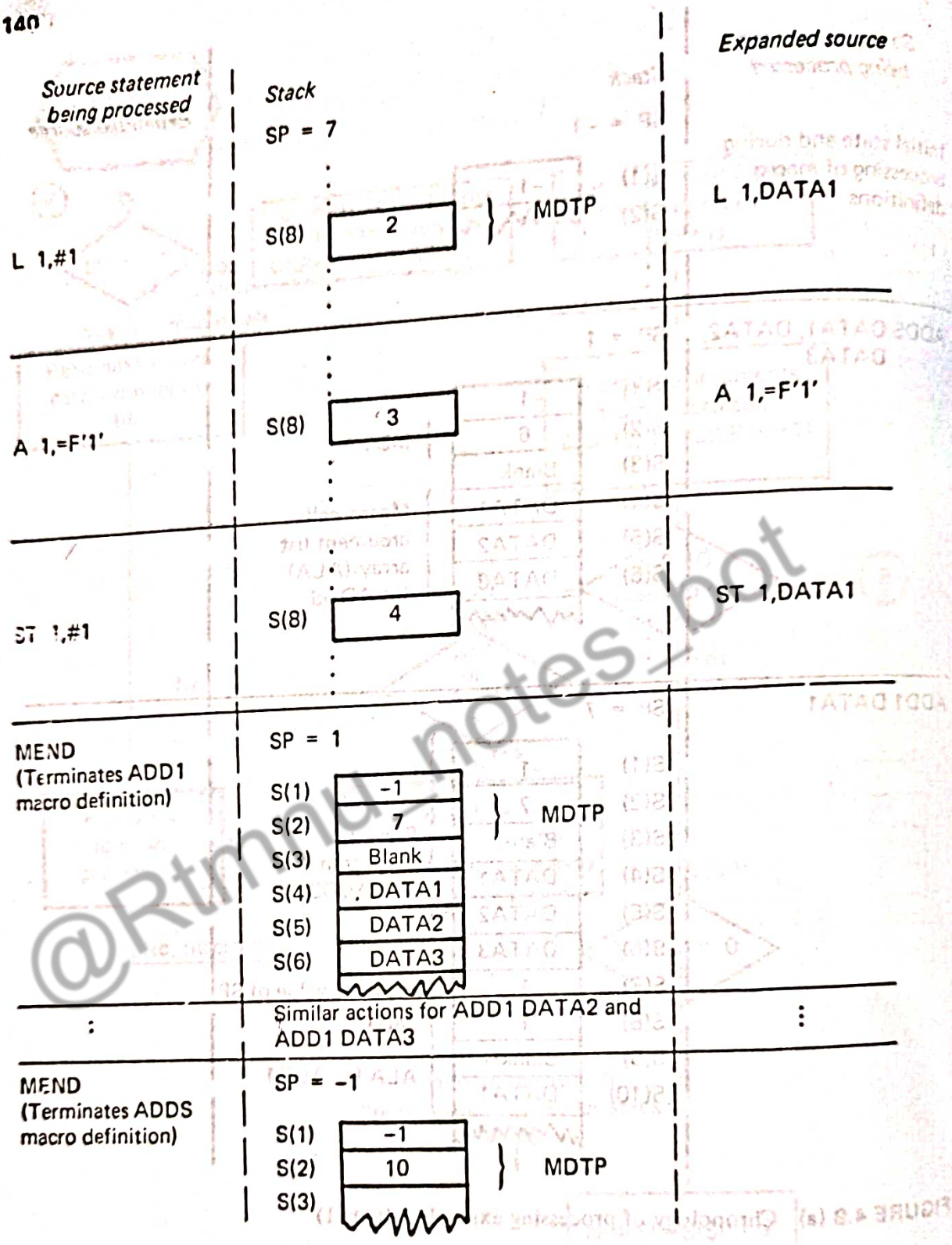


FIGURE 4.9 (b) Chronology of processing example 5 (part 2)

cessor and assembler by combining similar functions. For example, the Macro Name Table (MNT) can be combined with the assembler's op-code table (MOT or POT); a flag in each entry indicates whether or not it is a macro name. MACRO pseudo-ops to the macro processor can be detected by the assembler's

regular pseudo-op handler. The input READ function, which expands macro calls and receives the original source input, will be the same as that of Figure 4.8. Figure 4.10 is a flowchart of this algorithm; compare it with the flowcharts of Chapter 3.

The major advantages of incorporating the macro processor into pass 1 are:

1. Many functions do not have to be implemented twice (e.g., read a card, test for statement type).

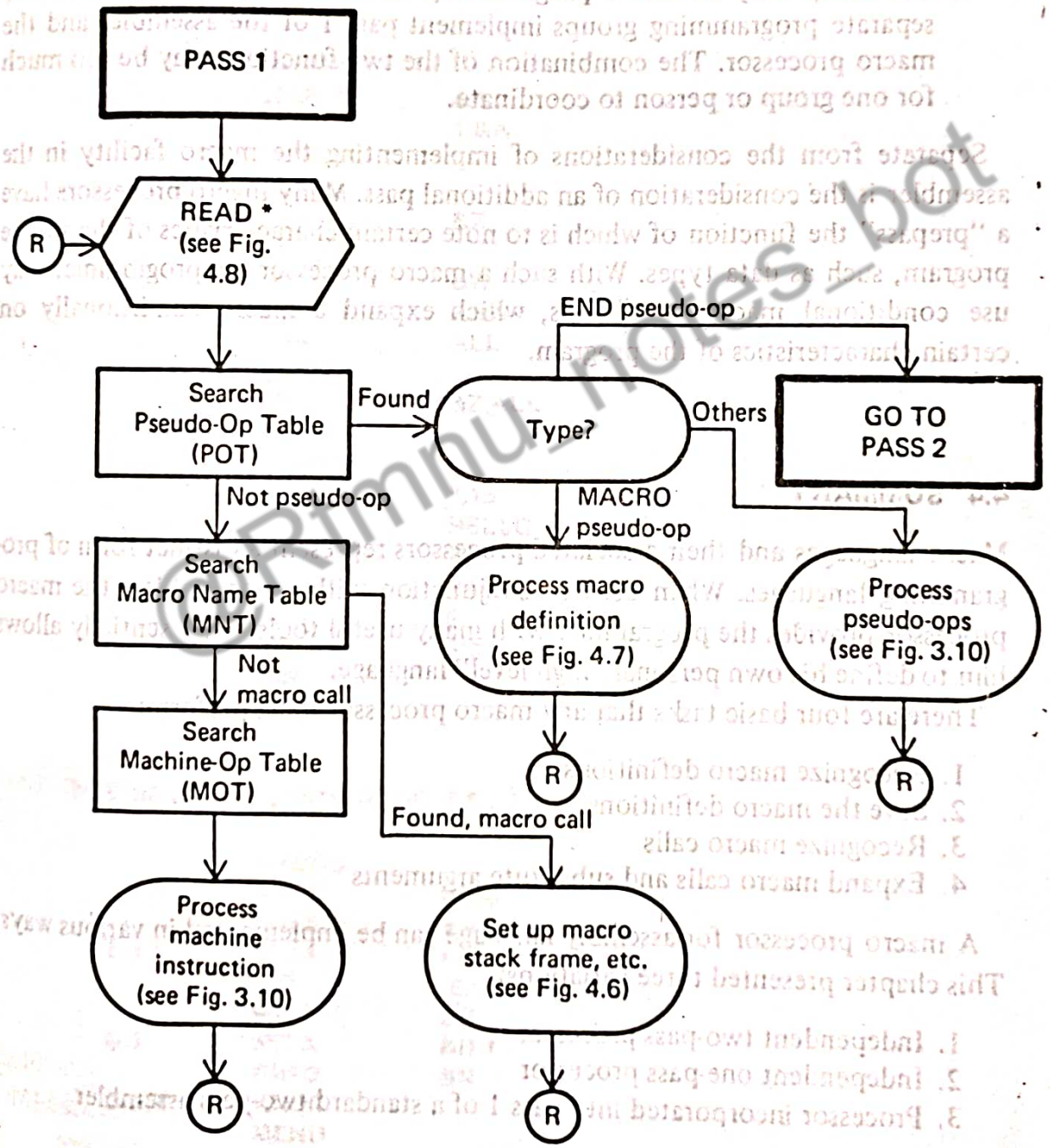


FIGURE 4.10 A macro processor combined with assembler pass 1

@Rmnu_notes_bot

2. There is less overhead during processing: functions are combined and it is not necessary to create intermediate files as output from the macro processor and input to the assembler.
3. More flexibility is available to the programmer in that he may use all the features of the assembler (e.g., EQU statements) in conjunction with macros.

The major disadvantages are:

1. The combined pass 1 of the assembler and the macro processor may be too large a program to fit into core of some machines.
2. The complexity of such a program may be overwhelming. Typically, two separate programming groups implement pass 1 of the assembler and the macro processor. The combination of the two functions may be too much for one group or person to coordinate.

Separate from the considerations of implementing the macro facility in the assembler is the consideration of an additional pass. Many macro processors have a "prepass" the function of which is to note certain characteristics of the source program, such as data types. With such a macro processor the programmer may use conditional macro pseudo-ops, which expand a macro conditionally on certain characteristics of the program.

@Rtmnu_notes_pdf

4.4 SUMMARY

Macro languages and their associated processors represent a distinct form of programming languages. When used in conjunction with an assembler, the macro processor provides the programmer with many useful tools and essentially allows him to define his own personal "high level" language.

There are four basic tasks that any macro processor must perform:

1. Recognize macro definitions
2. Save the macro definitions
3. Recognize macro calls
4. Expand macro calls and substitute arguments

A macro processor for assembly language can be implemented in various ways. This chapter presented three variations:

1. Independent two-pass processor
2. Independent one-pass processor
3. Processor incorporated into pass 1 of a standard two-pass assembler

QUESTIONS[1]

1. Describe the input and output of the macro processor. How dependent is it upon the assembler source code format?

2. For the following programs show:

   1) The expanded assembly language programming (i.e., no macros)
   2) The MDT table after macro processing
   3) The MNT table after processing

   The SETA and LCLA pseudo-op used in part b is as defined in problem 9.

   a.

```
        MACRO
        XYZ     &A
        ST      1,&A
        MEND
        MACRO
        MIT     &Z
        MACRO
        &Z      &W
        AR      4,&W
        XYZ     ALL
        MEND
        ST      &Z,ALL
        MEND
PROG    START
        USING   *,15
        MIT     HELLO
        ST      2,3
        HELLO   YALE
YALE    EQU     5
ALL     DC      F'3'
        END
```

   b. Computes exponentiation, e.g., 5^3

```
        MACRO
        EXPO    &EXP
        LCLA    &N
&N      SETA    &EXP
        AIF     (&N EQ 1) .STOP
        MR      0,2
&N      SETA    &N-1
        EXPO    &N
.STOP   ANOP
        MEND
```

[1] An * denotes that the question may require the use of IBM 360 manuals.

@Ptmnu_notes_bot

```
EXPON    START   0
         USING   *,15
         L       2,BASE
         L       1,BASE
         SR      0,0
         EXPO    3
         ST      1,ANS
         BR      14
ANS      DS      F
BASE     DC      F'5'
         END
```

3. The macro assembler presented in this chapter used its data bases in two different ways. It made references to the data (without making any modifications) and it created or adjusted the data (that is, it modified the data). In the matrix below, various data bases used in the assembler are listed in the vertical column. Across the top of the matrix are listed the various forms of data usage. Assume the macro processor has been combined with pass 1 of the assembler as in section 4.3.4.

	Must be built or modified in pass 1	Referenced but not modified in pass 1	Built during pass 1; saved for pass 2	Not modified during pass 2	Built or modified during pass 2	References occur in pass 2
Symbol table						
Literal table						
Base register table						
Collate tape						
Machine Op Table (MOT) includes MNT						
Pseudo-op table						
Input deck						
Macro Definition Table (MDT)						
Location counter						
Macro stack						

4. In a sense, macro expansion is very similar to subroutine calls during program execution. Explain the similarities and differences between them. What is the analog of the stack frame during subroutine calls?

5. a. If the assembler is processing a macro call and this macro contains a macro definition, would a new stack frame be set up? If so, what information would be stored?

@Rtmnu_notes_bot

b. Do any other data bases get changed in the handling of this nested definition? If so, which ones and how are they changed?
c. After processing the MEND card associated with the nested definition, can a macro call to this MACRO appear in the outer macro?
d. Could it appear anywhere in the outer macro? Could it appear anywhere in the program?
e. If we did not allow macro definitions within macro definitions, would we be able to remove the macro stack from the assembler? If so, why? If not, why not?
f. Which macro features required us to have a stack and the need for recursion?

6. If labels are allowed in the macro definition, what modification must be made to the macro processor implementation given in this chapter?

7. a. In order to be able to process macros in a single pass, we had to restrict the macro language. Describe the restriction and the limitations that it imposes on program organization.
b. Can a one-pass macro processor successfully handle a macro call with conditional macro pseudo-ops?

Consider the following case:

```
        MACRO
        WCM     &S
        .
        .
        AIF     (&S EQ 19).END
        .
        .
.END    MEND
        .
        .
        .
        WCM     V
        .
        .
        .
V       EQU     12
        .
        .
        .
        END
```

c. If a one-pass processor cannot, what modifications would be necessary to enable it to handle this type of situation?

8. a. We wish to implement an assembler using only a macro processor (i.e., the macro processor is to accept assembly language and output hexadecimal machine code). Consider a macro processor with *only* two pseudo-operations (SET,BYTE) and the two macro pseudo-ops MACRO and MEND.

@Rtmnu_notes_bot

The two pseudo-operations are.

(1)α SET β The symbol α in the label field is assigned the value of the expression β in the operand field. A symbol's value may be reset by another SET. Symbol LC is used as the assembler's Location Counter.

(2)BYTE expression (radix, size), [expression (radix, size)] . . . The BYTE pseudo-op will generate one 8-bit machine byte at the location corresponding to the current value of the Location Counter (@LC). The bytes are made up of subfields appearing from left to right in the operand field. The *radix* specifies whether the expression is in hex (16) or decimal (10); the *size* specifies the number of bits (out of 8) for that subfield. Example: Define AR-like instruction (add register to register); assume numeric op-code of hexadecimal '1A.'

```
        MACRO
&VAR1   AR      &VAR2, &VAR3
&VAR1   SET     LC
        BYTE    1A (16,8)
LC      SET     LC+1
        BYTE    &VAR2 (10,4), &VAR3 (10,4)
LC      SET     LC+1
        MEND
```

After 'AR' has been defined as a MACRO, this "assembler" can process AR instructions (i.e., accept an AR instruction and produce the equivalent hexadecimal machine instruction). Define SR (subtract register).

b. Define the IBM 360 instructions A, L (add, load – see Chapter 2).

9. It is often useful when constructing macros to save strings of characters for later use. In order to provide such a facility within the framework of an assembler like that discussed in this chapter we wish to add the following:

1) In addition to introducing dummy or "variable" symbols (those beginning with the character "&") in the prototype statement, it shall be possible to create variable symbols with two other statements: a GLOBAL statement and a LOCAL statement. The syntax of these statements is:

```
GBLA    &symbol
  or
LCLA    &symbol
```

where "&symbol" is a variable symbol called a "set symbol."

The GBLA (*GloBaL Arithmetic*) declares that the symbol has a value which is "global" in the sense that all macros which use the same name will reference the same value. It is much like declaring a label to

©Rtmnu_notes_bot

be an EXTRN in a running program although this "set symbol" only has a value during the macro processing phases of the assembler.

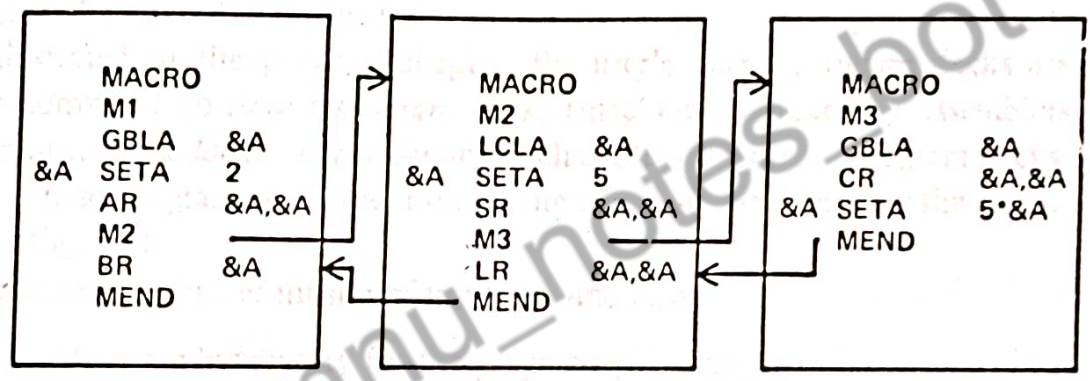
The LCLA (*LoCaL Arithmetic*) declares that the symbol has a value which is initialized to 0 each time the macro containing it is invoked, and that different macros containing "set symbols" of the same name will be referencing unique values.

An arithmetic symbol is stored and processed as a 32-bit fullword binary integer.

2) In order that we may assign values to "set symbols" we will create a statement of the form:

```
&set-symbol SETA arithmetic-expression
```

which will assign the value of the expression to the "set symbol."




```
.
.
.
25    M1
26+   AR    2,2
27+   SR    5,5
28+   CR    2,2
29+   LR    5,5
30+   BR    10
```

(+denotes a generated statement not in source code)

Make certain that you understand how the LCLA, GBLA and SETA pseudo-ops work in the above call of M1 before you answer the following questions.

For questions a, b, c, and d below you may assume that the macro processor was implemented in pass 1 of the assembler with a stack similar to that discussed in section 4.3.3.

a. 1) Which passes of the assembler must be modified for each pseudo-op?
   2) Which phases of the macro facility (definition or expansion) must be modified for each pseudo-op?

@Rtmnu_notes_bot

b. 1) Could *local* (LCLA) symbols be *efficiently* implemented by merely appending entries to the stack frame, much as macro parameters are handled? If not, how could they be handled? Explain both how your scheme would work and why you would choose it over the other one.

2) Answer the questions in b. 1) for global (GBLA) symbols.

c. Describe your method for processing the SETA instruction. Consider specifically both local and global symbols and how to update the correct variable.

d. 1) Describe how you would process an LCLA statement.

2) Do the same for the GBLA statement.

10. *Modify the macro processor designs given in this chapter (i.e., one-pass implementation, two-pass implementation, and assembler implementation) so that they can process AIF and AGO statements.



@Rtmnu_notes_bot